Mastering Design Patterns, Principles, and Architectures using .NET

BEGINNER TO ADVANCED
A structured path from design fundamentals to enterprise architecture

PRACTICAL .NET LEARNING
Real-world examples, UML guidance, refactoring, and ASP.NET Core implementations

LIVE TRAINING
Concept clarity, hands-on coding, design decisions, and direct guidance

Course Details

- **Course Duration: 2 Months (Monday to Friday), Daily 1:30 Hours**
- **Course Registration Link: https://forms.gle/vPceecJ5HpPUfrK3A**
- **Join our Telegram Group for more information: https://telegram.me/dotnettutorials**
- **Mobile / WhatsApp Number: +91 70218 01173**
- **WhatsApp Group: https://chat.whatsapp.com/BtH53ZmuopbFky0Om9pL3W**

Join Demo Sessions:

Date: 10th, 11th, and 12th August
Time: 06:30 AM – 08:00 AM, IST
Join Zoom Meeting
https://us06web.zoom.us/j/88960733536?pwd=6A2Ok8thw2GUNV0XvMwBLh3mzosDiM.1

Meeting ID: 889 6073 3536
Passcode: 381554

What You Will Gain

- A strong foundation in OOP, SOLID principles, Dependency Injection, unit testing with xUnit, layered design, MVC, and the Options Pattern.
- Practical understanding of all 23 Gang of Four patterns, including UML structure, real-world use cases, advantages, limitations, and implementation guidance.
- Hands-on knowledge of Repository, Unit of Work, Specification, Result, and CQRS patterns used in enterprise .NET applications.
- The ability to design applications using Clean Architecture and Domain-Driven Design without unnecessary overengineering.
- Confidence to identify design problems, compare alternatives, explain trade-offs, and select a solution that fits the project.

TRAINER
Pranaya Rout
Founder, Dot Net Tutorials
.NET Developer and Technical Trainer
Creator of practical, beginner-friendly .NET learning resources

REGISTER & CONNECT
WhatsApp: **+91 7021801173**
Email: **info@dotnettutorials.net**
Register for the Training Program
View Detailed Syllabus Online

Detailed Course Syllabus

Mastering Design Patterns, Principles, and Architectures using .NET

Course Roadmap

- Module 1: Software Design Foundations
- Module 2: .NET Application Design Foundations
- Module 3: GoF Design Patterns Roadmap
- Module 4: Creational Design Patterns
- Module 5: Structural Design Patterns
- Module 6: Behavioral Design Patterns
- Module 7: Enterprise Application Design Patterns
- Module 8: Architecture Patterns for Enterprise .NET Applications

Module 1: Software Design Foundations

This module establishes the design foundation required for the rest of the course. You will learn how object-oriented programming, SOLID principles, Dependency Injection, and unit testing work together to produce .NET applications that are easier to understand, test, extend, and maintain.

Chapter 1: Introduction to Software Design and Design Patterns

This chapter explains why software design matters and how poor design leads to rigid, duplicated, difficult-to-test code. You will learn what design patterns are, how they differ from algorithms, libraries, frameworks, and architecture, and how to recognize when a pattern adds value or introduces unnecessary complexity.

Topics Covered

- What is software design?
- Why software design is important in application development
- Problems caused by poor software design:
  - Rigid code
  - Duplicate logic
  - Difficult maintenance
  - Difficult testing
  - Low reusability
- What are Design Patterns?
- Why Design Patterns Were Introduced in Software Engineering
- History of Design Patterns and the Gang of Four
- Design Patterns vs:
  - Algorithms
  - Frameworks
  - Libraries
  - Architecture

- Why Design Patterns are needed in real-world applications
- Benefits of using Design Patterns:
  - Reusability
  - Flexibility
  - Maintainability
  - Cleaner code organization
- When to use Design Patterns
- When not to use Design Patterns
- Understanding overengineering and pattern misuse

Chapter 2: Object-Oriented Programming Concepts

This chapter revisits the object-oriented concepts required to understand design patterns correctly. You will learn how encapsulation, abstraction, inheritance, polymorphism, interfaces, abstract classes, and object relationships work together, and then apply these concepts in a practical ASP.NET Core Web API example.

Topics Covered

- What is Object-Oriented Programming?
- Why OOP is important for Design Patterns
- Classes and Objects
- Encapsulation:
  - Hiding internal state
  - Exposing controlled behavior
- Abstraction:
  - Showing what matters
  - Hiding unnecessary implementation details
- Inheritance:
  - Reusing behavior
  - Extending existing classes
- Polymorphism:
  - Multiple behaviors
  - Runtime flexibility
- Interface vs Abstract Class:
  - When to use an interface
  - When to use an abstract class
- Tight coupling vs loose coupling from an OOP perspective
- A Real-time .NET Core Web API application to understand the above concepts.

Chapter 3: SOLID Design Principles

This chapter explains the five SOLID principles as practical guidelines for designing maintainable object-oriented systems. Through analogies, before-and-after refactoring examples, and an ASP.NET Core mini application, you will learn how each principle reduces coupling, improves extensibility, and prepares the codebase for design patterns.

Topics Covered

- What are SOLID Design Principles?
- Why SOLID matters in real-world .NET projects
- Single Responsibility Principle:
  - What it means

    - Why one class should have one reason to change
    - Layman analogy to understand Single Responsibility Principle
    - Refactoring sample code for Single Responsibility Principle. First sample code without SRP and then refactor the sample code with SRP.
    - Real-world examples
- Open/Closed Principle:
    - Open for extension, closed for modification
    - Avoiding frequent code changes
    - Layman analogy to understand Open/Closed Principle
    - Refactoring sample code for Open/Closed Principle. First sample code without OCP and then refactor the sample code with OCP.
    - Real-world examples
- Liskov Substitution Principle:
    - Correct inheritance behavior
    - Problems caused by broken abstractions
    - Layman analogy to understand Liskov Substitution Principle
    - Refactoring sample code for Liskov Substitution Principle. First sample code without LSP and then refactor the sample code with LSP.
    - Real-world examples
- Interface Segregation Principle:
    - Small focused interfaces
    - Avoiding unnecessary method implementation
    - Layman analogy to understand Interface Segregation Principle
    - Refactoring sample code for Interface Segregation Principle. First sample code without ISP and then refactor the sample code with ISP.
    - Real-world examples
- Dependency Inversion Principle:
    - High-level modules should not depend on low-level modules
    - Depending on abstractions
    - Layman analogy to understand Dependency Inversion Principle
    - Refactoring sample code for Dependency Inversion Principle. First sample code without DIP and then refactor the sample code with DIP.
    - Real-world examples
- Implementing a SOLID-based mini application in .NET using ASP.NET Core Web API.

Chapter 4: Dependency Injection and Inversion of Control

This chapter explains how Inversion of Control and Dependency Injection remove direct object-creation dependencies from application code. You will learn the available injection techniques, .NET service lifetimes, registration and resolution, dependency graphs, and practical use of the built-in ASP.NET Core DI container.

Topics Covered

- What is Inversion of Control?
- What is Dependency Injection?
- Why direct object creation creates tight coupling?
- How Dependency Injection implements the Dependency Inversion Principle
- Benefits of Dependency Injection:
    - Loose coupling
    - Testability

    - Easier replacement of implementations
    - Better maintainability
- Types of Dependency Injection:
    - Constructor Injection
    - Property Injection
    - Method Injection
- DI lifetimes:
    - Transient
    - Scoped
    - Singleton
- How to Choose the correct lifetime?
- Built-in Dependency Injection container in .NET
- Service registration:
    - AddTransient
    - AddScoped
    - AddSingleton
- Service resolution and dependency graphs in .NET
- Real-time implementation in ASP.NET Core:
    - Service interface
    - Service implementation
    - Registration in DI container
    - Consumption in controller

Chapter 5: Unit Testing in .NET Using xUnit

This chapter introduces unit testing as an essential practice for building reliable, maintainable, and testable .NET applications. You will learn how to write automated tests using xUnit, organize tests using the Arrange-Act-Assert pattern, test different input scenarios, verify exceptions, and isolate business logic from external dependencies using mocking.

Topics Covered:

- Unit testing fundamentals
- Unit, integration, and end-to-end testing differences
- Arrange-Act-Assert pattern
- [Fact], [Theory], [InlineData], [MemberData], and [ClassData]
- Common xUnit assertions
- Synchronous and asynchronous testing
- Positive, negative, boundary, and exception scenarios
- Shared test context and fixtures
- Mocking dependencies using Moq
- Testing ASP.NET Core service-layer business logic
- Mocking repositories and Unit of Work
- A real-world Order Management Web API testing example

Module 2: .NET Application Design Foundations

This module translates core design principles into the structure of real ASP.NET Core applications. You will learn how to separate responsibilities into layers, understand the MVC request flow, and manage configuration through the Options Pattern in a clean and strongly typed manner.

Chapter 6: Layered Design in .NET Applications

This chapter introduces layered application design and explains how responsibilities should be separated across different parts of a .NET application. You will understand how layering improves maintainability, readability, testability, and scalability.

Topics Covered

- Why applications need layers
- Problems with putting all code in controllers or UI
- What is Layered Architecture?
- Common layers in .NET applications:
  - Presentation Layer
  - Application Layer
  - Domain Layer
  - Infrastructure Layer
- Responsibilities of each layer
- Dependency direction between layers
- Role of:
  - Controllers
  - Services
  - DTOs
  - Repositories
  - Domain models
- Where business logic should live
- Where validation should live
- Data access responsibilities
- Separation of concerns in real projects
- Layered structure vs tightly coupled structure
- Simple request flow through layers
- Real-time ASP.NET Core application structure
- Folder structure vs project structure
- Common layering mistakes
- Preparing for Clean Architecture later in the course

Chapter 7: MVC Pattern in ASP.NET Core

This chapter explains the MVC architectural pattern and how ASP.NET Core applies it to web application development. You will learn the role of Model, View, and Controller, understand request flow, and see how MVC helps create maintainable applications.

Topics Covered

- What is MVC?
- Why MVC exists
- Problems MVC solves
- Core MVC components:
  - Model
  - View
  - Controller
- Responsibility of each MVC component
- Understanding MVC request flow:
  - Request

    - Routing
    - Controller Action
    - Model interaction
    - View rendering
- Understanding MVC flow through a simple diagram
- Fat Controller vs Thin Controller
- Why business logic should not stay inside controllers
- Using services with MVC
- MVC and Dependency Injection
- MVC vs Web API:
    - Similarities
    - Differences
    - When to use each
- Implementing a small MVC workflow in ASP.NET Core

Chapter 8: Options Pattern in ASP.NET Core

This chapter teaches how to manage application configuration cleanly using the Options Pattern. You will learn how to bind settings from configuration files to strongly typed classes, validate them, and inject them into services safely.

Topics Covered

- Why configuration should not be scattered across the codebase
- Problems with hardcoded configuration values
- What is the Options Pattern?
- Why Options Pattern is preferred in ASP.NET Core
- Components involved:
    - Configuration source
    - Options class
    - DI registration
    - Consumer service
- Creating strongly typed options classes
- Binding configuration sections
- Understanding:
    - IOptions<T>
    - IOptionsSnapshot<T>
    - IOptionsMonitor<T>
- When to use each options interface
- Named Options:
    - Why they are needed
    - Multiple configurations of the same type
- Real-time examples:
    - JWT Settings
    - Email Settings
    - Cache Settings
    - Payment Gateway Settings
- Implementing Options Pattern in a .NET application
- Common mistakes while using Options Pattern

Module 3: GoF Design Patterns Roadmap

This module provides the big picture before pattern-by-pattern implementation begins. You will understand the Gang of Four pattern classification, basic UML concepts, and how to choose the right pattern for a design problem instead of memorizing patterns blindly.

Chapter 9: GoF Design Patterns Overview and Selection Guide

This chapter gives you a complete roadmap of the Gang of Four design patterns. You will understand how patterns are categorized, how UML diagrams help visualize them, and how to identify the most suitable pattern for a problem.

Topics Covered

- What are Gang of Four Design Patterns?
- Why GoF patterns are still relevant
- Three main categories:
  - Creational Patterns
  - Structural Patterns
  - Behavioral Patterns
- Difference between the three categories
- Complete list of GoF patterns
- High-level purpose of each pattern category
- Pattern relationships and combinations
- Why UML diagrams are useful for Design Patterns
- UML basics required in this course:
  - Class
  - Interface
  - Inheritance
  - Realization
  - Association
  - Aggregation
  - Composition
  - Dependency
- Reading a basic Design Pattern UML diagram
- How to select the right pattern for a problem
- Pattern selection checklist:
  - What varies?
  - What should stay stable?
  - Is creation, structure, or behavior the problem?
- Pattern selection examples from real applications
- Avoiding pattern memorization without understanding

Module 4: Creational Design Patterns

This module focuses on object creation. You will learn how to create objects in a controlled, reusable, and flexible way without tightly coupling the application to concrete implementations.

Chapter 10: Singleton Pattern

This chapter explains the Singleton Pattern, which ensures that only one instance of a class exists throughout the application lifecycle. You will learn why it is used, how it is implemented safely, and why modern .NET applications often prefer DI-managed singleton services.

Topics Covered

- What is the Singleton Pattern?
- Why the Singleton Pattern is needed
- Problems solved by Singleton
- Intent and motivation of Singleton
- Real-time scenarios where one shared instance is useful
- Key components involved:
  - Private constructor
  - Static instance
  - Public instance accessor
- Understanding UML diagram of Singleton Pattern
- Basic Singleton implementation
- Lazy initialization
- Eager initialization
- Thread-safe Singleton
- Double-checked locking concept
- Lazy<T> based Singleton
- Singleton pitfalls:
  - Hidden global state
  - Testing difficulties
  - Overuse
- Singleton vs static class
- Singleton vs DI Singleton lifetime
- When to avoid Singleton
- Real-time uses:
  - Configuration cache
  - Application-wide logger abstraction
  - Shared in-memory lookup provider
- Implementing Singleton in a real-time .NET example
- Best practices for modern .NET applications

Chapter 11: Factory Method Pattern

This chapter teaches the Factory Method Pattern, which delegates object creation to subclasses or specialized creator classes. You will learn how it reduces coupling to concrete classes and supports extensibility without changing existing code.

Topics Covered

- What is Factory Method Pattern?
- Why object creation should sometimes be delegated
- Problems caused by directly using new everywhere
- Intent and motivation of Factory Method
- Factory Method vs Simple Factory
- Components involved:
  - Product interface/base class
  - Concrete products
  - Creator abstraction
  - Concrete creators
- Understanding UML diagram of Factory Method Pattern

- How Factory Method supports Open/Closed Principle
- Encapsulating object creation logic
- Real-time use cases:
  - Notification sender creation
  - Payment processor creation
  - Export generator creation
- Implementing Factory Method in a real-time .NET application
- Example idea:
  - Create report exporters such as PDF, Excel, and CSV
- Advantages of Factory Method
- Limitations of Factory Method
- When Factory Method is appropriate
- When a simple approach is enough

Chapter 12: Abstract Factory Pattern

This chapter explains the Abstract Factory Pattern, which creates families of related objects without exposing concrete implementations. You will understand how this pattern helps maintain consistency when an application must support multiple platforms, providers, or product families.

Topics Covered

- What is Abstract Factory Pattern?
- Why Abstract Factory is needed
- Problems solved by Abstract Factory
- Intent and motivation
- Factory of factories concept
- Components involved:
  - Abstract factory
  - Concrete factories
  - Abstract products
  - Concrete products
  - Client
- Understanding UML diagram of Abstract Factory Pattern
- Creating families of related objects
- Preventing incompatible object combinations
- Abstract Factory vs Factory Method
- Real-time use cases:
  - UI themes
  - Database providers
  - Cloud storage providers
  - Payment gateway families
- Implementing Abstract Factory in a real-time .NET application
- Example idea:
  - Multi-provider notification system with Email and SMS families
- Advantages
- Drawbacks
- When Abstract Factory is useful
- When it may become unnecessary complexity

Chapter 13: Builder Pattern

This chapter introduces the Builder Pattern, which helps construct complex objects step by step. You will learn how it improves readability, avoids overloaded constructors, and makes object creation more expressive in real-world .NET applications.

Topics Covered

- What is Builder Pattern?
- Why Builder Pattern is needed
- Problems with telescoping constructors
- Intent and motivation
- Components involved:
  - Product
  - Builder interface or class
  - Concrete builder
  - Director, where applicable
- Understanding UML diagram of Builder Pattern
- Step-by-step object construction
- Optional and mandatory values
- Fluent Builder Pattern
- Builder with immutable objects
- Builder vs Factory
- Real-time use cases:
  - Email message construction
  - SQL query building
  - Invoice generation
  - API response object creation
- Implementing Builder in a real-time .NET application
- Example idea:
  - Building a complex order invoice object
- Advantages
- Limitations

Chapter 14: Prototype Pattern

This chapter explains the Prototype Pattern, which creates new objects by copying existing ones. You will learn how cloning works, when it is useful, and how to choose between shallow and deep copying in real-world applications.

Topics Covered

- What is Prototype Pattern?
- Why Prototype Pattern is needed
- When object cloning is useful
- Intent and motivation
- Components involved:
  - Prototype interface or base contract
  - Concrete prototype
  - Clone operation
- Understanding UML diagram of Prototype Pattern
- Creating objects through cloning
- Shallow copy vs deep copy

- Cloning nested objects
- Implementing Prototype in C#
- Manual clone vs helper-based approaches
- Real-time use cases:
  - Document templates
  - Product templates
  - Configuration copies
  - Game objects
- Implementing Prototype in a real-time .NET application
- Example idea:
  - Cloning a preconfigured notification template
- Advantages
- Limitations and practical concerns
- When Prototype is worth using

Module 5: Structural Design Patterns

This module teaches how to organize classes and objects into larger structures. You will learn how to connect incompatible systems, simplify complex subsystems, add behaviour dynamically, model hierarchies, control access, and optimize memory usage.

Chapter 15: Adapter Pattern

This chapter teaches the Adapter Pattern, which helps incompatible interfaces work together. You will understand how adapters are used to integrate legacy code, third-party libraries, and external services without changing core business logic.

Topics Covered

- What is Adapter Pattern?
- Why Adapter Pattern is needed
- Problems caused by incompatible interfaces
- Intent and motivation
- Components involved:
  - Target interface
  - Adaptee
  - Adapter
  - Client
- Understanding UML diagram of Adapter Pattern
- Object Adapter vs Class Adapter
- How Adapter protects core application design
- Real-time use cases:
  - Legacy payment provider integration
  - Third-party email service integration
  - Old API to new interface conversion
- Implementing Adapter Pattern in a real-time .NET application
- Example idea:
  - Adapting a legacy SMS provider to a modern notification interface
- Advantages
- Limitations
- Adapter vs Facade

Chapter 16: Facade Pattern

This chapter explains the Facade Pattern, which provides a simplified entry point to a complex subsystem. You will learn how it hides internal complexity and makes modules easier for other developers or application layers to use.

Topics Covered

- What is Facade Pattern?
- Why Facade Pattern is needed
- Problems solved by Facade
- Intent and motivation
- Components involved:
  - Facade class
  - Subsystem classes
  - Client
- Understanding UML diagram of Facade Pattern
- Simplifying a complex subsystem
- Hiding internal implementation details
- Providing a clean service entry point
- Facade vs Adapter
- Facade vs Service Layer
- Real-time use cases:
  - Payment checkout workflow
  - Order placement workflow
  - Report generation process
- Implementing Facade Pattern in a real-time .NET application
- Example idea:
  - OrderProcessingFacade coordinating inventory, payment, and notification services
- Advantages
- Limitations
- Where Facade fits in layered architectures

Chapter 17: Decorator Pattern

This chapter introduces the Decorator Pattern, which adds new behavior to an object without modifying its original class. You will understand how decorators provide flexible extension and how they are useful in logging, caching, validation, and retry scenarios.

Topics Covered

- What is Decorator Pattern?
- Why Decorator Pattern is needed
- Problems with extending behavior using inheritance only
- Intent and motivation
- Components involved:
  - Component interface
  - Concrete component
  - Base decorator
  - Concrete decorators
- Understanding UML diagram of Decorator Pattern
- Adding responsibilities dynamically

- Wrapping behavior around an existing service
- Decorator vs Inheritance
- Decorator vs Proxy
- Real-time use cases:
  - Logging decorator
  - Caching decorator
  - Validation decorator
  - Retry decorator
  - Auditing decorator
- Implementing Decorator Pattern in a real-time .NET application
- Example idea:
  - Decorating a ProductService with caching and logging
- Decorator with Dependency Injection
- Advantages
- Limitations

Chapter 18: Composite Pattern

This chapter explains the Composite Pattern, which models tree structures where individual objects and groups of objects can be treated uniformly. You will learn how this pattern simplifies hierarchical structures like menus, folders, and nested categories.

Topics Covered

- What is Composite Pattern?
- Why Composite Pattern is needed
- Problems with handling individual and group objects separately
- Intent and motivation
- Components involved:
  - Component
  - Leaf
  - Composite
  - Client
- Understanding UML diagram of Composite Pattern
- Tree-based object structures
- Treating individual and composite objects uniformly
- Managing recursive structures
- Real-time use cases:
  - File systems
  - Menu structures
  - Product category trees
  - Organizational hierarchies
- Implementing Composite Pattern in a real-time .NET application
- Example idea:
  - Category tree with parent and child categories
- Advantages
- Limitations
- When Composite is useful

Chapter 19: Proxy Pattern

This chapter teaches the Proxy Pattern, which controls access to another object through an intermediate wrapper. You will learn how proxies support lazy loading, authorization checks, caching, and remote object communication.

Topics Covered

- What is Proxy Pattern?
- Why Proxy Pattern is needed
- Intent and motivation
- Components involved:
  - Subject interface
  - Real subject
  - Proxy
  - Client
- Understanding UML diagram of Proxy Pattern
- Types of proxies:
  - Virtual Proxy
  - Protection Proxy
  - Remote Proxy
  - Caching Proxy
- Lazy loading through Proxy
- Access control and security scenarios
- Proxy vs Decorator
- Proxy vs Adapter
- Real-time use cases:
  - Image/document lazy loading
  - Authorization-protected service
  - Remote API wrapper
- Implementing Proxy Pattern in a real-time .NET application
- Example idea:
  - Secure document access proxy that checks permissions
- Advantages
- Limitations

Chapter 20: Bridge Pattern

This chapter explains the Bridge Pattern, which separates abstraction from implementation so both can evolve independently. You will understand how it avoids class explosion when a system has multiple independent dimensions of variation.

Topics Covered

- What is Bridge Pattern?
- Why Bridge Pattern is needed
- Problems caused by multiple inheritance-like variations
- Intent and motivation
- Components involved:
  - Abstraction
  - Refined abstraction
  - Implementor interface
  - Concrete implementors

- Understanding UML diagram of Bridge Pattern
- Separating abstraction from implementation
- Avoiding class explosion
- Bridge vs Strategy
- Bridge vs Adapter
- Real-time use cases:
    - Notifications over multiple channels
    - Remote control and device combinations
    - Report type and export format combinations
- Implementing Bridge Pattern in a real-time .NET application
- Example idea:
    - Notification type separated from delivery channel
- Advantages
- Limitations
- When to use Bridge Pattern

Chapter 21: Flyweight Pattern

This chapter introduces the Flyweight Pattern, which reduces memory consumption by sharing common object data. You will learn the difference between shared and external state and understand where this optimization is practically useful.

Topics Covered

- What is Flyweight Pattern?
- Why Flyweight Pattern is needed
- Memory issues caused by large numbers of similar objects
- Intent and motivation
- Components involved:
    - Flyweight interface
    - Concrete flyweight
    - Flyweight factory
    - Client
- Understanding UML diagram of Flyweight Pattern
- Intrinsic state vs extrinsic state
- Shared object reuse
- Flyweight Factory responsibility
- Real-time use cases:
    - Character rendering
    - Map markers
    - Reusable UI style objects
    - Shared product metadata references
- Implementing Flyweight Pattern in a real-time .NET application
- Example idea:
    - Reusing category style or icon configuration for many display items
- Performance considerations
- When Flyweight is useful
- When it is unnecessary
- Limitations

Module 6: Behavioral Design Patterns

This module explains how objects communicate, how responsibilities flow, and how application behavior can change cleanly without large conditional blocks. You will learn behavioral patterns that improve flexibility, collaboration, workflow handling, and event-driven logic.

Chapter 22: Strategy Pattern

This chapter teaches the Strategy Pattern, which enables an application to switch between multiple algorithms or business rules at runtime. You will learn how it removes large conditional blocks and supports extensibility.

Topics Covered

- What is Strategy Pattern?
- Why Strategy Pattern is needed
- Problems caused by large if-else or switch logic
- Intent and motivation
- Components involved:
  - Strategy interface
  - Concrete strategies
  - Context
  - Client
- Understanding UML diagram of Strategy Pattern
- Selecting algorithms dynamically
- Strategy and Open/Closed Principle
- Strategy vs State
- Real-time use cases:
  - Discount calculation
  - Payment fee calculation
  - Shipping cost rules
  - Tax calculation
- Implementing Strategy Pattern in a real-time .NET application
- Example idea:
  - Discount engine with seasonal, festival, and premium-user discounts
- Advantages
- Limitations

Chapter 23: State Pattern

This chapter explains the State Pattern, which changes an object's behavior based on its current state. You will learn how it simplifies state-driven workflows and removes complex conditional logic.

Topics Covered

- What is State Pattern?
- Why State Pattern is needed
- Problems caused by repeated state-based if-else logic
- Intent and motivation
- Components involved:
  - Context
  - State interface

    - Concrete states
- Understanding UML diagram of State Pattern
- State transitions
- Delegating behavior to state classes
- State vs Strategy
- Real-time use cases:
    - Order lifecycle
    - Ticket workflow
    - Loan approval process
    - Document approval process
- Implementing State Pattern in a real-time .NET application
- Example idea:
    - Order moves from Placed → Paid → Shipped → Delivered → Cancelled
- Advantages
- Limitations

Chapter 24: Template Method Pattern

This chapter introduces the Template Method Pattern, which defines the skeleton of an algorithm while allowing subclasses to customize specific steps. You will learn how it supports controlled reuse of process logic.

Topics Covered

- What is Template Method Pattern?
- Why Template Method is needed
- Intent and motivation
- Components involved:
    - Abstract base class
    - Template method
    - Primitive operations
    - Hook methods
    - Concrete subclasses
- Understanding UML diagram of Template Method Pattern
- Defining an algorithm skeleton
- Allowing selected steps to vary
- Fixed steps vs customizable steps
- Hook methods and extension points
- Template Method vs Strategy
- Real-time use cases:
    - Report generation workflow
    - File import workflow
    - Payment processing sequence
- Implementing Template Method in a real-time .NET application
- Example idea:
    - Data import template for CSV, Excel, and XML files
- Advantages
- Limitations
- Best practices

Chapter 25: Command Pattern

This chapter explains the Command Pattern, which encapsulates requests as objects. You will learn how it separates the request initiator from the request executor and supports workflows such as undo, logging, and queuing.

Topics Covered

- What is Command Pattern?
- Why Command Pattern is needed
- Problems solved by turning requests into objects
- Intent and motivation
- Components involved:
  - Command interface
  - Concrete command
  - Receiver
  - Invoker
  - Client
- Understanding UML diagram of Command Pattern
- Encapsulating requests as objects
- Decoupling sender and executor
- Command history
- Undo and redo behavior
- Queueing commands
- Logging executed commands
- Command vs Strategy
- Real-time use cases:
  - Order operations
  - Text editor undo
  - Background job execution
  - Transactional actions
- Implementing Command Pattern in a real-time .NET application
- Example idea:
  - Product inventory commands: Add, Update, Disable
- Advantages
- Limitations

Chapter 26: Chain of Responsibility Pattern

This chapter teaches the Chain of Responsibility Pattern, where a request passes through a sequence of handlers until it is processed. You will relate this pattern to ASP.NET Core middleware and learn how it supports extensible pipelines.

Topics Covered

- What is Chain of Responsibility Pattern?
- Why Chain of Responsibility is needed
- Intent and motivation
- Components involved:
  - Handler abstraction
  - Concrete handlers
  - Next handler reference
  - Client

- Understanding UML diagram of Chain of Responsibility Pattern
- Passing requests across multiple handlers
- Pipeline-based request processing
- Continue or stop execution logic
- Relationship with ASP.NET Core Middleware
- Real-time use cases:
  - Validation pipelines
  - Approval workflows
  - Request filtering
  - Authentication and authorization flow
- Implementing Chain of Responsibility in a real-time .NET application
- Example idea:
  - Loan application verification pipeline
- Advantages
- Limitations
- Best practices

Chapter 27: Observer Pattern

This chapter explains the Observer Pattern, which allows one object to notify multiple dependent objects when its state changes. You will understand event-driven communication and connect it with .NET events and delegates.

Topics Covered

- What is Observer Pattern?
- Why Observer Pattern is needed
- Intent and motivation
- Components involved:
  - Subject
  - Observer interface
  - Concrete observers
  - Notification mechanism
- Understanding UML diagram of Observer Pattern
- Subscribing and unsubscribing observers
- Broadcasting state changes
- Event-driven communication
- .NET events and delegates connection
- Observer vs Publish-Subscribe
- Real-time use cases:
  - Notification updates
  - Stock price tracking
  - UI refresh events
  - Order status notifications
- Implementing Observer Pattern in a real-time .NET application
- Example idea:
  - OrderPlaced event notifying email, SMS, and audit subscribers
- Advantages
- Limitations

Chapter 28: Mediator Pattern

This chapter introduces the Mediator Pattern, which centralizes communication between objects to reduce direct dependencies. You will learn how it simplifies collaboration in complex workflows and how the idea relates conceptually to MediatR.

Topics Covered

- What is Mediator Pattern?
- Why Mediator Pattern is needed
- Problems caused by many-to-many direct object communication
- Intent and motivation
- Components involved:
  - Mediator interface
  - Concrete mediator
  - Colleague objects
- Understanding UML diagram of Mediator Pattern
- Centralizing communication
- Reducing object coupling
- Coordinating complex workflows
- Mediator in UI systems
- Mediator in enterprise application workflows
- Conceptual relation to MediatR
- Real-time use cases:
  - Chat room coordination
  - Booking workflow coordination
  - Form component interaction
- Implementing Mediator Pattern in a real-time .NET application
- Example idea:
  - Order checkout coordinator communicating with payment, inventory, and notification services
- Advantages
- Limitations

Chapter 29: Iterator Pattern

This chapter explains the Iterator Pattern, which provides a standard way to traverse collections without exposing their internal structure. You will understand how custom traversal logic can be implemented cleanly.

Topics Covered

- What is Iterator Pattern?
- Why Iterator Pattern is needed
- Intent and motivation
- Components involved:
  - Iterator
  - Concrete iterator
  - Aggregate
  - Concrete aggregate
- Understanding UML diagram of Iterator Pattern
- Traversing collections safely
- Hiding internal collection details

- Forward and custom traversal
- Iterator vs foreach
- Iterator concept in .NET collections
- Real-time use cases:
  - Paginated item traversal
  - Custom tree traversal
  - Playlist navigation
- Implementing Iterator Pattern in a real-time .NET application
- Example idea:
  - Navigating a custom product catalog collection
- Advantages
- Limitations

Chapter 30: Memento Pattern

This chapter teaches the Memento Pattern, which captures and restores an object's previous state without exposing its internal implementation. You will understand how it supports undo, rollback, and history-based scenarios.

Topics Covered

- What is Memento Pattern?
- Why Memento Pattern is needed
- Intent and motivation
- Components involved:
  - Originator
  - Memento
  - Caretaker
- Understanding UML diagram of Memento Pattern
- Saving object state
- Restoring previous state
- Preserving encapsulation
- Undo operations
- Rollback mechanisms
- Version history
- Real-time use cases:
  - Text editor undo
  - Form draft recovery
  - Workflow rollback
- Implementing Memento Pattern in a real-time .NET application
- Example idea:
  - Restoring previous invoice draft state
- Advantages
- Limitations
- Performance considerations

Chapter 31: Visitor Pattern

This chapter explains the Visitor Pattern, which allows new operations to be added to an existing object structure without modifying the classes in that structure. You will learn why Visitor is powerful, how double dispatch works, and when this pattern becomes useful.

Topics Covered

- What is Visitor Pattern?
- Why Visitor Pattern is needed
- Intent and motivation
- Components involved:
  - Visitor interface
  - Concrete visitors
  - Element interface
  - Concrete elements
  - Object structure
- Understanding UML diagram of Visitor Pattern
- Adding new operations without modifying existing objects
- Double dispatch concept
- Traversing object structures with visitors
- Visitor vs adding methods directly into classes
- Real-time use cases:
  - Tax calculation across product types
  - Exporting different entity structures
  - Report generation
- Implementing Visitor Pattern in a real-time .NET application
- Example idea:
  - Calculating shipping or tax differently for product types
- Advantages
- Drawbacks
- When Visitor is useful
- When Visitor is overkill

Chapter 32: Interpreter Pattern

This chapter introduces the Interpreter Pattern, which defines a grammar and evaluates expressions based on that grammar. You will understand how it supports simple rule engines and why it becomes difficult for complex language processing.

Topics Covered

- What is Interpreter Pattern?
- Why Interpreter Pattern is needed
- Intent and motivation
- Components involved:
  - Abstract expression
  - Terminal expression
  - Non-terminal expression
  - Context
- Understanding UML diagram of Interpreter Pattern
- Expression evaluation
- Grammar representation
- Parsing simple rules
- Building rule trees
- Real-time use cases:
  - Simple search filters
  - Discount expression evaluation

    - Permission rule evaluation
- Implementing Interpreter Pattern in a real-time .NET application
- Example idea:
    - Basic promotion rule parser such as “amount > 5000 AND customerType = Premium”
- Advantages
- Practical limitations
- Why complex parsing usually needs dedicated tools

Module 7: Enterprise Application Design Patterns

This module focuses on enterprise patterns commonly used in real-world .NET applications. You will understand how to structure data access, organize filtering logic, manage business outcomes consistently, and separate read and write responsibilities.

Chapter 33: Repository, Generic Repository, and Unit of Work

This chapter explains how repositories separate business logic from data access and how Unit of Work manages transactions consistently. You will also understand where these patterns help, where they are misused, and how they fit with EF Core.

Topics Covered

- What is Repository Pattern?
- Why Repository Pattern is used
- Problems caused by mixing data access with business logic
- Repository as an abstraction over persistence
- Components involved:
    - Repository interface
    - Concrete repository
    - Domain or service consumer
- Understanding repository flow through a simple architecture diagram
- Generic Repository Pattern:
    - Why it is used
    - Common CRUD abstraction
- Limitations of Generic Repository
- What is Unit of Work?
- Why Unit of Work is needed
- Managing multiple repository operations in one transaction
- Unit of Work and transaction consistency
- Repository + Unit of Work with EF Core
- Are repositories always needed with EF Core?
- Best practices
- Common anti-patterns
- Real-time .NET implementation:
    - Product repository
    - Order repository
    - Unit of Work for checkout or booking transaction

Chapter 34: Specification Pattern

This chapter introduces the Specification Pattern, which helps encapsulate reusable business rules and filtering logic. You will learn how it prevents query duplication and keeps repositories and services cleaner.

Topics Covered

- What is Specification Pattern?
- Why Specification Pattern is needed
- Problems caused by repeated query conditions
- Encapsulating business rules
- Components involved:
  - Specification abstraction
  - Concrete specifications
  - Specification evaluator
  - Repository consumer
- Understanding specification flow diagram
- Reusable query logic
- Combining specifications:
  - AND
  - OR
  - NOT
- Using Specification with Repository Pattern
- Real-time use cases:
  - Active products
  - Premium customers
  - Available hotels
  - Orders within date range
- Implementing Specification Pattern in a .NET application
- Example idea:
  - Product filtering by category, price range, and stock status
- Benefits
- Limitations
- When Specification Pattern is useful

Chapter 35: Result Pattern

This chapter explains the Result Pattern, which provides a clean way to represent success and failure outcomes without using exceptions for expected business scenarios. You will learn how it improves consistency in services, APIs, and CQRS handlers.

Topics Covered

- What is Result Pattern?
- Why Result Pattern is needed
- Problems with exception-driven control flow
- Result Pattern for expected business failures
- Components involved:
  - Success flag
  - Message
  - Error code
  - Error list

    - Value in Result<T>
- Result vs Result<T>
- Validation errors using Result Pattern
- Mapping Result to HTTP responses:
    - 200 OK
    - 400 Bad Request
    - 404 Not Found
    - 409 Conflict
- Chaining results
- Early returns for cleaner service logic
- Using Result Pattern in application services
- Using Result Pattern inside CQRS handlers
- Handling domain validation and application validation
- Real-time use cases:
    - Registration workflow
    - Payment result
    - Booking confirmation
- Implementing Result Pattern in a .NET Web API
- Example idea:
    - Create order service returning structured success/failure response
- Best practices for consistent error handling
- Limitations

Chapter 36: CQRS Pattern

This chapter teaches the CQRS Pattern, which separates read operations from write operations. You will understand why this separation improves clarity, flexibility, and scalability in certain applications, and when CQRS is unnecessary.

Topics Covered

- What is CQRS?
- Why CQRS is needed
- Problems solved by separating commands and queries
- Commands vs Queries
- Read Model vs Write Model
- Components involved:
    - Command
    - Command handler
    - Query
    - Query handler
    - Dispatcher or mediator, where applicable
- Understanding CQRS flow diagram
- Validation in command handlers
- Returning optimized query models
- Real-time use cases:
    - Product catalog
    - Order management
    - Booking systems
- Implementing CQRS in a real-time .NET Web API
- Example idea:

    - CreateProductCommand and GetProductByIdQuery
- Benefits of CQRS
- When CQRS should not be used
- Common overengineering mistakes

Module 8: Architecture Patterns for Enterprise .NET Applications

This module teaches how to structure large .NET applications using proven architecture styles. You will compare Layered, Clean, Onion, Hexagonal, and DDD-based approaches and understand how to choose the right architecture based on project size and complexity.

Chapter 37: Clean Architecture

This chapter explains Clean Architecture, which organizes applications around business rules rather than frameworks or databases. You will learn the dependency rule, layer responsibilities, and how to structure ASP.NET Core applications in a maintainable way.

Topics Covered

- What is Clean Architecture?
- Why Clean Architecture is needed
- Problems with framework-centric application design
- Dependency Rule
- Domain-centric design
- Core layers:
    - Domain
    - Application
    - Infrastructure
    - Presentation
- Responsibilities of each layer
- Direction of dependencies
- Entities and use cases
- DTOs and application contracts
- Repository interfaces and implementations
- Infrastructure isolation
- Understanding Clean Architecture dependency diagram
- Clean Architecture with ASP.NET Core
- Real-time project structure
- Example idea:
    - Product Management API using Clean Architecture
- Benefits
- Limitations
- When Clean Architecture is appropriate
- Common mistakes

Chapter 38: Domain-Driven Design Architecture

This chapter introduces Domain-Driven Design as an approach for building software around real business concepts. You will learn both strategic and tactical DDD concepts and understand how DDD fits into complex enterprise applications.

Topics Covered

- What is Domain-Driven Design?
- Why DDD is needed
- When simple CRUD design becomes insufficient
- Strategic DDD vs Tactical DDD
- Ubiquitous Language
- Bounded Context
- Core DDD building blocks:
  - Entities
  - Value Objects
  - Aggregates
  - Aggregate Roots
  - Repositories
  - Domain Services
  - Domain Events
- Understanding aggregate boundary diagrams
- Entity vs Value Object
- Aggregate consistency rules
- Domain behavior vs anemic models
- DDD with Clean Architecture
- DDD with CQRS
- DDD with events
- Real-time business domains:
  - E-commerce
  - Hotel booking
  - Banking
  - Learning management
- Implementing a small DDD-focused .NET domain model
- Example idea:
  - Order Aggregate with OrderItems and domain behavior
- When DDD is beneficial
- When DDD is overkill
- Common beginner mistakes in DDD

By the End of This Course, You Should Be Able To

- Recognize recurring design problems and explain the forces and trade-offs involved.
- Refactor tightly coupled code using OOP, SOLID principles, and Dependency Injection.
- Choose and implement an appropriate GoF pattern without forcing patterns into simple problems.
- Apply enterprise patterns to organize data access, business rules, results, and application workflows.
- Compare layered design, Clean Architecture, Domain-Driven Design, enterprise architecture approaches.
- Evaluate architectural boundaries, dependencies, and trade-offs for maintainable enterprise applications.
- Communicate architectural decisions clearly and justify why a selected approach fits the project.

Conclusion

This course provides a complete learning path from foundational software design concepts to advanced enterprise architecture. It is designed to help .NET developers move beyond writing code that merely works and begin creating solutions that remain understandable, testable, maintainable, and adaptable as business requirements evolve.

By completing the course, you will understand not only how to implement design patterns, but also why they exist, which problems they solve, and when a simpler solution is more appropriate. You will be able to connect SOLID principles, Dependency Injection, unit testing, Gang of Four patterns, enterprise patterns, CQRS, Clean Architecture, and Domain-Driven Design within practical ASP.NET Core applications.

The ultimate goal is to develop strong design judgment. Rather than applying patterns mechanically, you will learn to analyze requirements, recognize design problems, evaluate trade-offs, and select an architecture that matches the size, complexity, scalability, and maintainability needs of the application.

Demo Sessions:

Please attend the Demo Sessions on 10th, 11th, and 12th August using the below Zoom Credentials.

Join Zoom Meeting
https://us06web.zoom.us/j/88960733536?pwd=6A2Ok8thw2GUNV0XvMwBLh3mzosDiM.1

Meeting ID: 889 6073 3536
Passcode: 381554
Date: 10/11/12 August 2026
Time: 06:30 AM – 08:00 AM IST

Ready to strengthen your software design and architecture skills?

Register for the Training Program

Join Our Learning Community

🌐 **Website: www.dotnettutorials.net**
✉️ **Email: info@dotnettutorials.net**
📞 **WhatsApp/Call: +91 70218 01173**
💬 **WhatsApp Group: Click here to join**
📢 **Telegram Group: @dotnettutorials**
📺 **YouTube Channel: Click here to join**

Share This Learning Material

Found this chapter helpful? Share it with your friends, colleagues, students, and anyone interested for this training program.

Keep Learning. Keep Practicing. Keep Growing.